The Mathematics behind Modern AI

Demystifying the Black Box

Ann-Sophie Hilkert Varga

July 2, 2026

The Core Idea

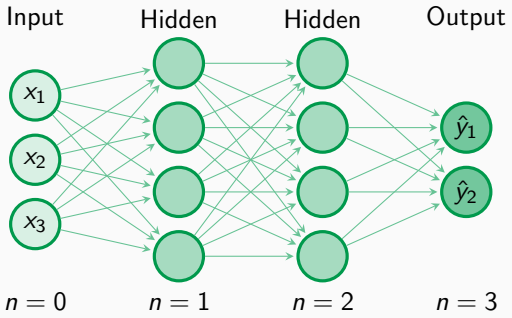
AI can be viewed as a parameterized function:

$$AI(x) = f(x; \theta)$$

where

- x = input
- f = model
- θ = learned parameters

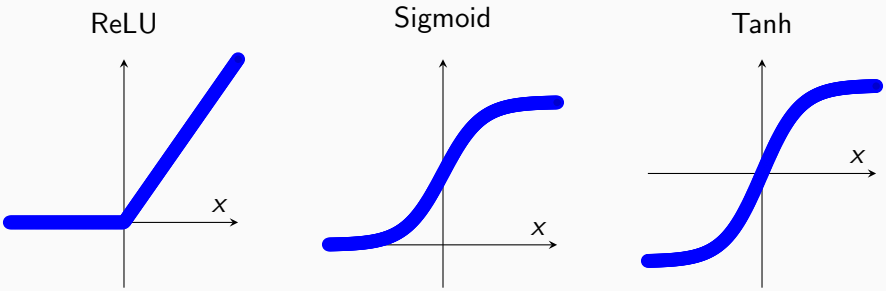
A Multilayer Perceptron



Edges represent the weights θ; nodes represent the activation outputs $\sigma(\theta x + b)$.

$$f(x) = f_n(f_{n-1}(\dots f_1(x)))$$

Common Activation Functions



Each introduces nonlinearity in a different way.

How Does Learning Happen?

We define a loss function:

$$L(\theta)$$

The goal:

$$\theta^* = \arg\min_{\theta} L(\theta)$$

Training = minimizing prediction error.

Gradient Descent

Suppose we are standing on a hill.

The gradient tells us which direction points uphill.

To go downhill:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla L$$

where

- $\eta =$ learning rate
- $\nabla L =$ gradient

Backpropagation

Question:

How do we compute gradients for millions or billions of parameters?

Answer:

The chain rule.

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

Backpropagation efficiently computes derivatives throughout the computational graph.

Convolutional Neural Networks (CNNs)

Specialized for data with spatial structure – above all, **images**.

Instead of connecting every pixel to every neuron, a CNN slides a small **filter** (kernel) across the image:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n)\, K(m, n)$$

- **Local:** each neuron looks only at a small patch.
- **Weight sharing:** the same filter is reused everywhere.
- Far fewer parameters than a fully connected layer.

Why CNNs Work So Well

Filters learn a **hierarchy of features**:

- Early layers: edges, corners, colors.
- Middle layers: textures, shapes, parts.
- Deep layers: whole objects (faces, cars, ...).

Key ingredients:

- **Convolution** – detect a feature anywhere in the image.
- **Pooling** – shrink the map, keep what matters.
- **Translation invariance** – a cat is a cat, wherever it appears.

The Transformer: Attention is all you need

A Transformer processes a whole sequence of patches at once.

The central idea: **attention** lets every patch look at every other patch and decide what is relevant.

Using a **Q**uery, **K**ey and **V**alue for each patch:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Query, Key, Value

Each patch embedding e is projected into three vectors:

- **Query** $Q = \theta_Q\, e$ – what am I looking for?
- **Key** $K = \theta_K\, e$ – what do I contain?
- **Value** $V = \theta_V\, e$ – what do I pass on?

The **learned weights live in the matrices** $\theta_Q, \theta_K, \theta_V$ (plus an output projection θ_O).

Attention itself has no parameters – training only adjusts these projection matrices.

The AI Training Loop

1. Receive input
2. Make prediction
3. Measure error
4. Compute gradients
5. Update parameters

New Architectures

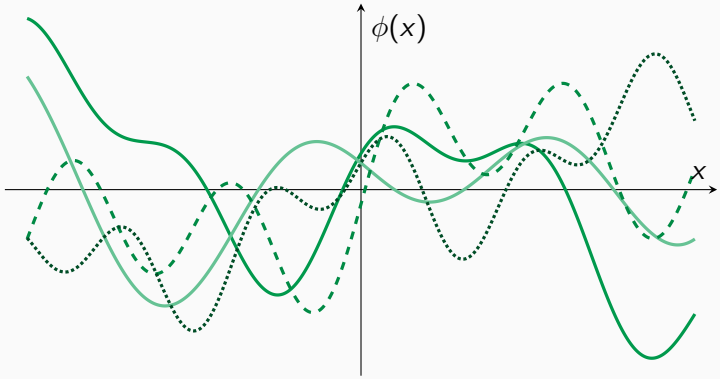
So far: parameters live on *edges* (weights θ), with fixed activations σ on nodes.

Kolmogorov–Arnold Networks (KANs) flip this idea:

$$f(x_1, \ldots, x_n) = \sum_q \Phi_q \left(\sum_p \phi_{q,p}(x_p) \right)$$

- Based on the Kolmogorov–Arnold representation theorem.
- *Learnable functions* $\phi_{q,p}$ (splines) sit on the edges.
- Nodes simply sum, no fixed activation function.

What Do Learnable Splines Look Like?



Each edge learns its own flexible curve $\phi_{q,p}(x)$, fitted via spline control points.

Other Paradigms

Not every model learns by gradient-based weight updates:

- **KANs:** learn activation functions instead of weights, more interpretable.
- **Spiking neural networks:** event-driven, biologically inspired dynamics.
- **Evolutionary / genetic algorithms:** optimize via mutation and selection, no gradients.
- **Bayesian models:** update probability distributions, not point estimates.
- **Symbolic / neuro-symbolic AI:** reason with explicit rules and logic.

Thank You

Questions?